МИРЭА
Российский технологический университет

Файловый ввод-вывод

Мухортова Н.Н.

Цель

Познакомиться и освоить системные вызовы ввода/вывода (input/output, I/O).

Познакомиться и освоить операторы ввода/вывода, встроенные в язык C.

В этой лекции обсуждаются системные вызовы нижнего уровня.

API и ABI

На системном уровне существует два отдельных множества определений и описаний, которые влияют на переносимость программ м/у различными дистрибутивами Linux. Одно из этих множеств называется интерфейсом программирования приложений (Application Programming Interface, API), а другое — двоичным интерфейсом приложения (Application Binary Interface, ABI). Обе эти концепции определяют и описывают интерфейсы между различными компонентами программного обеспечения.

API

API определяет интерфейсы, на которых происходит обмен информацией между двумя компонентами программного обеспечения на уровне исходного кода. API обеспечивает абстракцию, предоставляя стандартный набор интерфейсов — как правило, это функции, — которые один программный компонент (обычно, но не обязательно это более высокоуровневый компонент из пары) может вызывать из другого (обычно более низкоуровневого).

ABI

ABI предназначен для определения двоичного интерфейса между двумя и более программными компонентами в конкретной архитектуре. ABI определяет, как приложение взаимодействует с самим собой, с ядром и библиотеками. В то время как API обеспечивает совместимость на уровне исходного кода, ABI отвечает за совместимость. Это означает, что фрагмент объектного кода будет функционировать в любой системе с таким же ABI без необходимости перекомпиляции.

Что такое файл

Файл представляет собой последовательность байтов, никак не организованных операционной системой. Прикладной программе файл представляется непрерывной последовательностью байтов (это не означает, что соответствующие байты занимают непрерывное пространство на физическом устройстве).

Не существует разницы между файлами, представляющими двоичные данные и текстовые данные. Ваша программа несет ответственность за преобразование (если оно необходимо) внешнего формата представления данных в требуемое представление. Например, функция atoi(3) удобна для преобразование чисел из текстового вида во внутреннее машинное двоичное представление.

Что такое файл

Каждый байт регулярного файла адресуется индивидуально. Вновь созданный файл не содержит пространства для данных. Пространство под данные предоставляется при записи в дисковый файл.

Система выделяет блоки физического диска под данные по мере необходимости. Размер файла хранится в структуре данных, называемой inode. В самом файле не присутствует признака конца файла.

Что такое файл

Операционная система UNIX рассматривает файл, как универсальный интерфейс с физическим устройством. Многие системные вызовы, применимые к файлам, могут быть применены и к байт- или блок-ориентированным устройствам. Однако некоторые вызовы, наподобие lseek(2), которые изменяют позицию ввода/вывода, неприменимы к файлам некоторых устройств, например, терминальных устройств.

Рассмотрим следующую программу:

```
main()
{
        printf("hi world\n");
}
```

В ней вызывается printf(3C), который, в свою очередь, вызывает write(2) от файлового дескриптора 1. Программа не потребует изменений, если вывод перенаправляется в файл или на другой терминал. Система UNIX ищет устройство или файл, ассоциированное с дескриптором 1, и вызывает требуемый драйвер устройства

Стандартные функции ввода/вывода

Стандартные библиотечные функции ввода/вывода (stdio), реализованные как обёртка над системными вызовами, предоставляют больше возможностей и большую функциональность.

Например, эти функции позволяют вводить и выводить:

- символ
- строку
- данные с преобразованием формата многочисленные структуры, содержащие двоичные и/или текстовые данные

Библиотечные функции ввода/вывода уменьшают количество системных вызовов за счёт буферизации вводимых и выводимых данных, поэтому эти функции называют функциями буферизованного ввода/вывода.

Буферизованный ввод-вывод

Схема буферизации предполагает перенос данных из буфера сегмента данных программы во вторичный буфер. Вторичный буфер по умолчанию имеет размер BUFSIZ, определенный в <stdio.h>.

Когда вторичный буфер становится полным при выводе или пустым при вводе, производится необходимый системный вызов.

Буферизованный ввод-вывод

Системный вызов write(2) производит запись вторичного буфера в системный буфер, который, в конце концов, попадает на устройство вывода. read(2), в свою очередь, переносит данные из устройства ввода в системный буфер, оттуда — в пользовательский буфер.

Из этого буфера символы попадают в обычно меньший по размерам буфер, предоставляемый стандартными библиотечными функциями ввода/вывода.

Библиотечные функции ввода/вывода

1. Буфер будет выведен, только если он полон, или при вызове fflush(3C). Это называется блочной буферизацией
2. Файловые потоки могут быть строчно-буферизованы. Тогда буфер выводится при записи символа новой строки ASCII LF (в языке C этот символ обозначается как '\n'), при полном буфере или при запросе ввода. По умолчанию, вывод на терминал является строчно-буферизованным. Стандартный поток вывода (stdout) является строчно-буферизованным, если файловый дескриптор 1 назначен на терминал, и блочно-буферизованным, если файловый дескриптор 1 назначен на регулярный файл или какое-то другое устройство.
3. Файловые потоки могут быть небуферизованы. Стандартное устройство диагностики stderr небуферизовано.

Библиотечные функции ввода/вывода

Функция setbuf(3S) позволяет сделать потоки небуферизованными или переключить режим буферизации. Функция setvbuf(3S) также позволяет изменить размер буфера.

В общем, лучше использовать не системные вызовы ввода/вывода, а библиотечные функции. Во многих случаях это эффективнее, в частности — при переносе небольших порций данных.

Использование системных вызовов может быть целесообразно в ситуациях, когда вам необходимо гарантировать, что операции ввода/вывода и другие системные вызовы будут исполнены в определённой последовательности, например, при синхронизации с другим процессом.

Даже в таких ситуациях может быть удобнее выключить буферизацию на уровне библиотеки или использовать fflush(3C)

Открытие файла

Системный вызов open(2) открывает файл, то есть ассоциирует файловый дескриптор с обычным или специальным файлом.

Файл должен быть открыт до того, как будет осуществляться чтение или запись.

Файловый дескриптор используется для идентификации файла при обращении к другим системным вызовам. Файловый дескриптор представляет собой небольшое неотрицательное целое число.

Аргументы open(2)

path указывает на строку символов, содержащую путевое имя файла.

Имя может быть либо абсолютным (относительно корневого каталога), либо относительным (относительно текущего каталога).

Длина имени ограничена параметром PATH_MAX, значение которого можно получить вызовом pathconf(2) с параметром _PC_PATH_MAX.

Также, файловая система может накладывать ограничения на длину базового имени файла. Значение этого ограничения можно определить вызовом pathconf(2) с параметром _PC_NAME_MAX.

Аргументы open(2)

oflag указывает, в каком режиме следует открыть файл. Например, следует ли файл открыть для чтения и/или записи, создать и т.д.

Значение этого флага получается с помощью побитового ИЛИ символьных констант, определённых в <fcntl.h>

mode используется для установки битов прав доступа при создании файла. Внимание: ненулевое значение umask может изменить (уменьшить) права доступа, указанные в mode.

Файловый дескриптор

Значение, возвращаемое open(2), является наименьшим доступным файловым дескриптором от нуля до административного предела.

Этот дескриптор используется системными вызовами read(2) и write(2), а также рядом других системных вызовов и некоторыми библиотечными функциями.

При запуске процесса из стандартных командных оболочек, таких, как sh, ksh, bash, файловые дескрипторы 0, 1 и 2 уже определены при старте программы. В других контекстах, процесс может запускаться с другими наборами предопределенных дескрипторов. Файловый дескриптор имеет смысл либо до закрытия вызовом close(2), либо до завершения программы. Файловые дескрипторы разных процессов могут иметь одинаковые значения, но при этом указывать на разные файлы.

Файловый дескриптор

Максимальный номер файлового дескриптора на единицу меньше, чем максимально допустимое для процесса количество одновременно открытых файлов (в стандарте POSIX это ограничение обозначается OPEN_MAX). Значение OPEN_MAX ограничено «мягким» и «жестким» пределами. Мягкий (административный) предел устанавливается setrlimit(2) с командой RLIMIT_NOFILE или командой ulimit(1). Жёсткий предел устанавливается настройками ядра системы. Значение жёсткого предела можно определить системным вызовом sysconf(2) с параметром _SC_OPEN_MAX. В Solaris, жесткий предел устанавливается параметром rlim_fd_max в файле /etc/system (system(4)); его изменение требует административных привилегий и перезагрузки системы.

Файловый дескриптор

В Solaris 10, максимальное значение rlim_fd_max равно MAXINT (2147483647). По умолчанию, на 32-битной версии Solaris, OPEN_MAX равен 256, так как 32-битная версия стандартной библиотеки ввода-вывода использует байт для хранения номера дескриптора. Также, на 32-битной версии Solaris, используемая по умолчанию версия select(3C) поддерживает не более 1024 дескрипторов. Функция select(3C) обсуждается в разделе «Мультиплексирование ввода-вывода». В 64-битной версии Solaris, rlim_fd_max по умолчанию равен 65536. 64-битная версия select(3C) поддерживает не более 65536 дескрипторов

open(2) - Флаги

Следующие флаги, определенные в , могут быть объединены с помощью побитового ИЛИ и использованы совместно в аргументе oflag вызова open(2). Замечание: должен быть использован хотя бы один из первых трех флагов, иначе вызов open(2) окончится неуспехом.

O_RDONLY Открывает файл для чтения.

O_WRONLY Открывает файл для записи.

O_RDWR Открывает файл для чтения и для записи.

O_APPEND Перед каждой записью помещает указатель файла в конец файла. Иными словами, все операции записи будут происходить в конец файла.

O_CREAT Создает файл, если он не существует.

O_TRUNC Стирает данные файла, устанавливая размер файла равным нулю.

O_EXCL Используется совместно с

O_CREAT. Вызывает неуспех open(2), если файл уже существует.

O_SYNC Заставляет write(2) ожидать окончания физической записи на диск.

O_NDELAY,O_NONBLOCK Для некоторых устройств (терминалов, труб, сокетов), чтение может приводить к блокировке процесса, если в буфере устройства нет данных. Установка этих флагов приводит к переводу устройства в режим опроса. В этом режиме, чтение из устройства с пустым буфером возвращает управление немедленно.

O_NOCTTY Не позволяет терминальному устройству стать управляющим терминалом сессии.

open(2) - Флаги

Чтобы открыть файл с флагом O_RDONLY, необходимо иметь право чтения из этого файла, а с флагом O_WRONLY — соответственно, право записи. Права доступа проверяются только в момент открытия, но не при последующих операциях с файлом. Без использования флага O_CREAT, open(2) открывает существующий файл и возвращает ошибку ENOENT, если файла не существует. При использовании флага O_CREAT, open(2) пытается открыть существующий файл с указанным именем, и пытается создать такой файл, если его не было.

open(2) - Флаги

Аргумент mode должен быть использован совместно с флагом O_CREAT при создании нового файла. Права доступа у вновь открытого файла будут установлены в непредсказуемое значение, если mode будет опущен. Биты прав доступа устанавливаются в значение mode & ~umask, где umask — значение полученное после выполнения команды umask(1). Обратите внимание, что аргумент mode объявлен как необязательный, поэтому компилятор C не выдаёт синтаксическую ошибку при его отсутствии.

open(2) - Флаги

Флаги O_NDELAY и O_NONBLOCK воздействуют на именованные программные каналы и специальные байт-ориентированные файлы. Использование флага O_NDELAY при открытии терминального файла и различие между O_NDELAY и O_NONBLOCK обсуждаются далее в этом разделе. Их воздействие на программные каналы обсуждается в разделе «Программные каналы»

Права доступа к файлу

Маска прав доступа в Unix представляет собой 12-разрядную битовую маску, которую удобно записывать в виде 4- или 3-разрядного восьмеричного числа.

Старшие 3 бита (восьмеричные тысячи) кодируют setuid, setgid и sticky-биты.

Следующие 9 бит кодируют собственно права доступа.

Биты с 9 по 7 (восьмеричные сотни) кодируют права доступа хозяина файла, биты с 6 по 4 (восьмеричные десятки) — права группы, младшие три бита с 3 по 1 (восьмеричные единицы) — права всех остальных пользователей.

Старший бит в тройке (4 в восьмеричной записи) кодирует право чтения, средний бит (2 в восьмеричной записи) — право записи, младший бит (1 в восьмеричной записи) — право исполнения.

Права доступа к файлу

Порядок бит, кодирующих права, соответствует порядку символов, описывающих права, в выдаче команды ls -l и ряда других стандартных утилит Unix.

Так, запись -rw-r—r-- в выдаче ls -l соответствует битовой маске 0644 и означает, что хозяин имеет право чтения и записи, а группа и все остальные — только права чтения.

Права доступа к файлу

Атрибут процесса cmask (creation mask), который может быть установлен системным вызовом umask(2), представляет собой 9-битовую маску.

При создании файла, в маске, указанной при вызове open(2), биты, соответствующие битам, установленным в cmask, очищаются. Так, cmask 0022 очищает биты права записи для группы и всех остальных и оставляет в неприкосновенности остальные биты.

Таким образом, cmask позволяет задавать права доступа к файлам по умолчанию или, точнее, ограничения на права по умолчанию.

Типичные значения cmask, применяемые на практике — 0022 (пользователь сохраняет все права, группа и все остальные не имеют права записи) и 0027 (пользователь сохраняет все права, группа не имеет права записи, все остальные не имеют никаких прав).

Примеры

Приведенные ниже объявления необходимы для перечисленных на этой странице примеров. Включение файла необходимо для использования символьных имен флагов open(2).

```
#include <fcntl.h>

#define TMPFILE "/tmp/tmpfile"

char account[] = "account";

int logfd, acctfd, fd, fdin, fdout; char *file;
```

Примеры

Файл account, находящийся в текущем каталоге, открывается для чтения.

acctfd = open(account, O_RDONLY);

Файл, на имя которого указывает file, открывается для записи. Если файл не существует, он создается с маской прав доступа 0600 (возможно, модифицированной umask). Если файл существует, он будет усечен до нулевого размера.

file = TMPFILE;

fd = open(file, O_WRONLY | O_CREAT | O_TRUNC, 0600)

Примеры

Файл с абсолютным путевым именем ("/sys/log") открывается для записи. Все записи производятся в конец файла. Если файл не существует, он создаётся с правами доступа 0600.

```
logfd = open("/sys/log", O_WRONLY | O_APPEND | O_CREAT, 0600);
```

Файл, имя которого было передано main как первый аргумент, открывается на чтение/запись.

```
fdin = open(argv[1], O_RDWR);
```

Примеры

Файл открывается на запись. Если он не существует, он создается. Если файл существовал, вызов open(2) будет завершен неуспешно. Заметьте, что код возврата open(2) проверяется в условии оператора if. Программа должна предпринять некие действия в случае неуспеха; в данном примере она распечатывает код ошибки.

```
if ((fdout = open(TMPFILE, O_WRONLY | O_CREAT | O_EXCL, 0666)) == -1)
        perror(TMPFILE);
```

Примеры

Флаг O_EXCL используется для предотвращения непреднамеренной перезаписи уже существующего файла. Используется совместно с O_CREAT.

Вызов open(2) окончится неуспехом, если файл уже существует. Этот флаг не означает, что программа открыла файл исключительно для себя, или что это единственный файл, открытый программой.

Вызов open(2)

Информация об открытых файлах хранится в ядре UNIX. Таблица файловых дескрипторов, размещенная в пользовательской области процесса, содержит указатели на системные структуры, описывающие файл. Сами эти структуры могут разделяться несколькими процессами, и поэтому находятся в разделяемой памяти ядра за пределами пользовательской области. Файловый дескриптор — это число в диапазоне от 0 до MAX_OPEN-1, которое является индексом в таблице файловых дескрипторов.

Системные файловые структуры содержат информацию о конкретном открытом файле. Для каждого вызова open(2) выделяется собственная структура, даже если два разных вызова открыли один и тот же файл на диске.

В структуре содержится

.**указатель на текущую позицию в файле**. Этот указатель изменяется на прочитанное/записанное количество байт при каждом вызове read(2) и write(2). Кроме того, позицию в файле можно установить явно вызовом lseek(2).

. **копия флагов открытия.** Эти флаги передаются вторым аргументом open(2).

. **счетчик ссылок.** Это число различных файловых дескрипторов из одной или различных пользовательских областей, которые совместно используют данную системную структуру.

Процесс может создавать новые дескрипторы, указывающие на имеющиеся структуры (ранее открытые файлы), системным вызовом dup(2). Кроме того, при создании нового процесса, он наследует все открытые родителем файлы — это соответствует ссылкам на одну структуру из разных пользовательских областей.

. **указатель на структуру информации о файле** (образ инода в памяти).

Структура информации о файле

.номер устройства, на котором размещен файл,

. пользовательский идентификатор владельца файла и идентификатор группы файла.

. счетчик ссылок - количество файловых дескрипторов, ссылающихся на данную структуру.

. связи – количество записей в директориях, указывающих на данный файл.

. тип файла – обычный, директория, специальный файл и пр. для специальных файлов устройств - «старшее» (major) и «младшее» (minor) числа и указатель на минорную запись устройства; «Старшее» число идентифицирует драйвер, а «младшее» - номер устройства, управляемого этим драйвером. Минорная запись (minor record) — системная структура данных, описывающая устройство, и содержащая блок переменных состояния устройства и указатели на функции драйвера.

Структура информации о файле

. права доступа данного файла.

. размер файла в байтах. временные штампы создания файла, последней модификации и последнего доступа к файлу.

. список номеров блоков для блоков данных на диске

Закрытие файла

Системный вызов close(2) служит для закрытия файла, то есть для уничтожения файлового дескриптора. После вызова close(2), файловый дескриптор становится недействительным.

Когда все файловые дескрипторы, указывающие на системную структуру данных, закрываются, структура данных также уничтожаются.

close(2) освобождает системные ресурсы. Если программы не закрывает файл, это производится системой при завершении работы программы, например, при вызове exit(2).

Закрытие файла

Если ваша программа использует много файловых дескрипторов, например, для работы с сокетами TCP/IP, необходимо закрывать неиспользуемые дескрипторы, иначе ваш процесс может столкнуться с нехваткой дескрипторов.

Также, закрытие сокетов, труб и некоторых специальных файлов может иметь другие побочные эффекты. Так, закрытие дескриптора сокета TCP приводит к разрыву TCP-соединения, то есть к освобождению ресурсов как на локальном компьютере, так и на том узле сети, с которым было установлено соединение

Чтение из файла

Чтение из файла осуществляется системными вызовами read(2) и readv(2). read(2) читает байты в единый буфер, в то время как readv(2) позволяет осуществлять разбросанное чтение в несколько несмежных буферов одним вызовом.

Аргументы для этих вызовов: fildes файловый дескриптор, полученный при предшествующем вызове open(2) с флагами O_RDONLY или O_RDWR.

Чтение из файла

buf указывает на место, в которое должны быть помещены прочитанные байты. Места должно хватить для nbyte байт.

nbyte максимальное число байт, которые следует считать. На самом деле может быть прочитано меньшее число байт.

iov указывает на массив структур **struct iovec** со следующими полями:

- caddr_t iov_base;
- int iov_len;

Чтение из файла

Каждая структура iovec содержит адрес и длину области памяти, куда будут помещены байты вызовом readv(2).

Вызов заполняет последовательно указанные области памяти, переходя от одной к другой.

iovcnt количество структур iovec.

Чтение из файла

Значение, возвращаемое read(2) и readv(2) показывает количество байт, прочитанных на самом деле. Например, в файле осталось 10 байт, а nbyte равен 15.

Значение, возвращаемое read(2) будет 10. Если read(2) будет вызвана вновь, она вернет 0. Это индикация конца файла.

Чтение из файла

Чтение из специального байт-ориентированного файла также может возвращать меньшее число байт, чем требуемое. Так, по умолчанию, терминалы буферизуют ввод по строкам (эта буферизация осуществляется драйвером терминального устройства; не следует путать ее с буферизацией библиотеки ввода-вывода).

При чтении, драйвер терминала возвращает одну строку, даже если в буфере есть еще символы.

Чтение из труб или сокетов, как правило, возвращает те данные, которые находятся в приемном буфере на момент вызова read(2), и не ожидает прихода дополнительных данных.

Файл-сокет

Сетевой сокет-это программная структура в сетевом узле компьютерной сети, которая служит конечной точкой для отправки и приема данных по сети.

Файлы - трубы или каналы

Pipe используется для передачи вывода в другую программу или утилиту .

Redirect используется для передачи вывода в файл или поток .

Пример: thing1 > thing2 против thing1 | thing2

thing1 > thing2

Ваша оболочка запустит программу с именем thing1

Все, что thing1 выводится, будет помещено в файл с именем thing2. (Примечание - если thing2 существует, он будет перезаписан)

Файлы - трубы или каналы

Pipe используется для передачи вывода в другую программу или утилиту .

Redirect используется для передачи вывода в файл или поток .

Пример: thing1 > thing2 против thing1 | thing2

thing1 > thing2

Ваша оболочка запустит программу с именем thing1

Все, что thing1 выводится, будет помещено в файл с именем thing2. (Примечание - если thing2 существует, он будет перезаписан)

Файлы - трубы или каналы

Если вы хотите передать вывод из программы thing1в вызываемую программу thing2, вы можете сделать следующее:

thing1 > temp_file && thing2 < temp_file

которые бы

- запустить программу с именем thing1
- сохранить вывод в файл с именем temp_file
- Запустите программу с именем thing2, делая вид, что человек на клавиатуре напечатал содержимое в temp_fileкачестве ввода.

Однако, это неуклюже, поэтому они сделали трубы как более простой способ сделать это. thing1 | thing2делает то же самое, что иthing1 > temp_file && thing2 < temp_file

Чтение из файла

Значение nbyte никогда не должно превышать размера буфера buf. Нарушение этого условия может привести к срыву буфера — опасной ошибке программирования, которая может быть использована для исполнения произвольного кода с привилегиями вашего процесса, то есть для внедрения вирусов, червей и других вредоносных программ.

Чтение из файла

При чтении из регулярных файлов и из специальных файлов устройств, поддерживающих lseek(2), вызовы read(2) и readv(2) перемещают указатель чтения-записи. Это может быть неудобно в многопоточных программах или при работе с разделяемым файловым дескриптором из нескольких процессов. В таких ситуациях рекомендуется использовать вызов pread(2). У этого вызова есть дополнительный параметр off_t offset, указывающий позицию в файле, откуда следует начать чтение. pread(2) не перемещает указатель чтения-записи

Запись в файл

Системные вызовы write(2) и writev(2) записывают данные в открытый файл. write(2) записывает данные из единого буфера, а writev(2) позволяет осуществлять запись из нескольких несмежных областей памяти одним вызовом.

Аргументы

fildes файловый дескриптор, полученный при предшествующем вызове open(2) с флагами O_WRONLY или O_RDWR.

buf буфер с записываемыми байтами. Этот аргумент может быть указателем или именем массива.

nbyte максимальное число байт, которые следует записать. На самом деле, write может записать меньше байт, чем запрошено.

iov указывает на массив структур struct iovec, со следующими полями: caddr_t iov_base; int iov_len;

Каждая структура iovec содержит адрес и длину области памяти, откуда будут записаны байты вызовом writev(2). iovcnt количество структур iovec.

Запись в файл

Значение, возвращаемое write(2) и writev(2), показывает количество на самом деле записанных байт. Если достигнут предел размера файла (см. ulimit(2)), исчерпана дисковая квота или место на физическом устройстве, количество записанных байт будет меньше, чем nbyte.

Запись в файл

Как и read(2), при работе с регулярными файлами и устройствами, поддерживающими lseek(2), вызовы write(2) и writev(2) перемещают указатель чтения-записи.

В случаях, когда это нежелательно, рекомендуется использовать вызов pwrite(2), аналогичный вызову pread(2).

Если вам нужно гарантировать, чтобы все записи всегда происходили в конец файла (например, если несколько потоков или процессов пишут сообщения в один лог-файл), следует использовать при открытии файла флаг O_APPEND

Запись в файл

Поскольку read(2) и write(2) являются системными вызовами, их использование ведет к накладным расходам, связанным со всеми системными вызовами. При работе с дисковыми файлов эти вызовы наиболее эффективны, если данные считываются блоками размером 512 байт или более (на практике, чем больше блок, тем лучше). Для чтения/записи одиночных байт, небольших структур данных или коротких строк эффективнее использовать функции буферизованного ввода-вывода.

Копирование ввода в вывод

Программа копирует данные из стандартного ввода, файловый дескриптор 0, в стандартный вывод, файловый дескриптор 1. Файловые дескрипторы 0, 1 и 2 (стандартный вывод диагностики) открываются при запуске программы из стандартных командных процессоров. Обычно они связаны с терминальным устройством. Это можно изменить, используя перенаправление ввода/вывода в командной строке.

Эта программа работает следующим образом

файл <stdio.h> содержит определение BUFSIZ.

buf объявлен достаточно большим для вызова read(2)

требуется прочесть BUFSIZ байт.

Истинное число считанных байт присваивается переменной n и, как правило, равно BUFSIZ. n может быть меньше BUFSIZ, если читается конец файла файла или если ввод осуществляется с терминала. Если данных больше нет, возвращается 0, что указывает конец файла. В каждой итерации цикла записывается n байт. Для запуска программы наберите:

$ stdcopy <file1> file2

Файл: stdcopy.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

main()
{
    char buf[BUFSIZ];
    int n;

    while ((n = read(0, buf, BUFSIZ)) > 0)
        write(1, buf, n);
    exit(0);
}
```

Копирование файла

Этот пример похож на предыдущий, но в этом случае копирование осуществляется из одного файла в другой. Этот пример похож на команду cp(1).

13-17 Проверка правильности числа аргументов

18-21 Первый аргумент программы – имя копируемого файла, который открывается на чтение

22-26 Второй аргумент – имя файла, открываемого на запись. Если файл не существует, он будет создан. Иначе он будет усечен до нулевого размера. PMODE - символьная константа, используемая для установки битов прав доступа к файлу. В «настоящей» команде cp следовало бы копировать права доступа старого файла, но мы будем проходить чтение битов прав доступа только в разделе «Управление файлами»

```
 2 #include <sys/types.h>
 3 #include <stdlib.h>
 4 #include <unistd.h>
 5 #include <fcntl.h>
 6 #define   PMODE     0644
 7
 8 main(int argc, char *argv[])
 9 {
10     int fdin, fdout, n;
11     char buf[BUFSIZ];
12
13     if (argc != 3) {
14     fprintf(stderr, "Usage: %s filein fileout\n",
15         argv[0]);
16     exit(1);
17     }
18     if ((fdin = open(argv[1], O_RDONLY)) == -1) {
19     perror(argv[1]);
20     exit(2);
21     }
22     if ((fdout = open(argv[2], O_WRONLY | O_CREAT |
23     O_TRUNC, PMODE)) == -1 ) {
24     perror(argv[2]);
25     exit(3);
26     }
27
28     while ((n = read(fdin, buf, BUFSIZ)) > 0)
29     write(fdout, buf, n);
30
31     exit(0);
32 }
```

Создание файла информации о служащих

Программа создает "записи" о служащих. Структура employee определена в файле employee.h

Каждой записи предшествует заголовок, содержащий дату/время создания записи и идентификатор пользователя, создавшего запись. Заголовок определен в файле empheader.h

```
1 #define NAMESIZE  24
2
3 struct employee {
4    char name[NAMESIZE];
5    int salary;
6 };
```

```
1 struct recheader {
2      char date[24];
3      uid_t uid;
4 };
5 static void init_header(struct recheader *);
```

```
 8 #include <sys/uio.h>
 9 #include "employee.h"
10 #include "empheader.h"
11
12 main(int argc, char *argv[])
13 {
14     int fd;
15     struct employee record;
16     struct recheader header;
17     struct iovec iov[2];
...
23     if ((fd = open(argv[1], O_WRONLY | O_CREAT |
24     O_SYNC | O_EXCL, 0640)) == -1) {
25     perror(argv[1]);
26     exit(2);
27     }
28
29     iov[0].iov_base = (caddr_t)&header;
30     iov[1].iov_base = (caddr_t)&record;
31     iov[0].iov_len = sizeof(header);
32     iov[1].iov_len = sizeof(record);
33
34     init_header(&header);
35
36     for (;;) {
37     printf("Enter employee name <SPACE> salary: ");
38     scanf("%s", record.name);
39     if (record.name[0] == '.')
40         break;
41     scanf("%d", &record.salary);
42     writev(fd, iov, 2);
43     }
44     close(fd);
45     exit(0);
46 }
```

Ожидание физической записи на диск

По умолчанию, Unix использует для работы с файлами отложенную запись. Системные вызовы write(2), writev(2) и pwrite(2) завершаются после переноса данных в системные буферы и не ожидают завершения физической записи на устройство. При использовании флага O_SYNC при открытии файла, система будет работать в режиме прямой или сквозной записи. При этом, вызовы write(2) будут ожидать физического завершения записи.

Ожидание физической записи на диск

Если ожидание завершения физической записи необходимо только для некоторых операций, вместо флага O_SYNC можно использовать системный вызов fsync(2). Этот вызов завершается после переноса всех ожидавших записи данных, связанных с указанным файловым дескриптором, на физический носитель.

Ожидание физической записи на диск

fsync(2) может быть использован программами, которым необходимо, чтобы файл к определенному моменту находился в заданном состоянии. Например, программа, которая содержит простейшие возможности выполнения транзакций, должна использовать fsync(2), чтобы гарантировать, что все модификации файла или файлов, осуществленные в процессе транзакции, были записаны на носитель, прежде чем оповещать пользователя о завершении транзакции.

Ожидание физической записи на диск

Системный вызов sync(2) запрашивает запись на диск всех ожидающих записи блоков в системе (всего содержимого дискового кэша), но может вернуться прежде, чем эти операции будут завершены. Кроме пользовательских данных, дисковый кэш содержит модифицированные суперблоки, модифицированные иноды и запросы отложенного ввода-вывода других процессов. Этот вызов должен использоваться программами, которые анализируют файловую систему, и не рекомендуется к использованию прикладными программами общего назначения. Система автоматически производит запись данных из дискового кэша на диск, что, в определенном смысле, соответствует периодическому вызову sync(2).

Создание копии дескриптора файла

Вызов dup(2) дублирует файловый дескриптор. Новый файловый дескриптор ассоциируется с тем же файлом, имеет ту же позицию в файле и те же права доступа, что и fildes. dup(2) всегда возвращает наименьший возможный (или, что то же самое, первый свободный) файловый дескриптор. Вызов dup2(2) также дублирует дескриптор, но начинает поиск свободного файлового дескриптора не с 0, как dup(2), а с указанного номера.

Создание копии дескриптора файла

Основная задача dup(2) - обеспечить перенаправление ввода/вывода. Действительно, чтобы переназначить стандартный поток вывода (дескриптор 1), командный процессор должен закрыть старый дескриптор 1 и открыть новый файл. Если открытие нового файла по какойто причине завершится неудачей, запускаемый процесс рискует остаться вовсе без стандартного потока вывода. Чтобы избежать этого, командные процессоры сначала открывают новый файл, затем, если файл открылся успешно, закрывают старый стандартный поток вывода и только потом переназначают новый файл на поток вывода вызовом dup(2).

Что делает dup(2)

Системный вызов dup(2) копирует указатель на системную файловую структуру в таблице дескрипторов файлов в новую ячейку таблицы. Это позволяет двум файловым дескрипторам совместно использовать ту же самую позицию указателя, поскольку они указывают на одну и ту же системную файловую структуру.

Ядро поддерживает счетчик количества файловых дескрипторов, ссылающихся на системную структуру открытого файла. Закрытие каждого из дескрипторов приводит к уменьшению значения этого счетчика. При закрытии последнего дескриптора, счетчик становится равным нулю, и структура данных уничтожается

Перенаправление ввода/вывода

Программа переназначает стандартный вывод и стандартный вывод диагностики в указанный файл или устройство, приведенное в качестве первого аргумента командной строки. В командном процессоре такое перенаправление производится так:

prog >file 2>&1 или

prog 2>file 1>&2

Когда командный процессор замечает символ &, за которым следует десятичная цифра, он интерпретирует цифру как номер дескриптора файла и вызывает dup(2).

Перенаправление ввода/вывода

Библиотечная функция printf(3S) вызывает write(2) с файловым дескриптором 1. printf(3S) эквивалентен fprintf(3S), использующему stdout.

Используя stderr в качестве первого аргумента, fprintf вызовет write(2) с дескриптором 2.

```
 1 #include <unistd.h>
 2 #include <stdlib.h>
 3 #include <fcntl.h>
 4 #include <stdio.h>
 5
 6 /* demonstrate dup(2) */
 7
 8 main(int argc, char *argv[])
 9 {
10
11     close(1);
12     if (open(argv[1], O_WRONLY |
13     O_CREAT | O_TRUNC, 0644) == -1) {
14     perror(argv[1]);
15     exit(1);
16     }
17
18     close(2);
19     if (dup(1) == -1)
20     exit(2);
21
22     printf("first line to stdout (uses fd 1)\n");
23     fprintf(stderr,"first line to stderr (uses fd 2)\n
24     printf("second line to stdout\n");
25     fprintf(stderr,"second line to stderr\n");
26 }
```

Вывод

Все операции ввода/вывода осуществляются системными вызовами или библиотечными функциями, которые, в свою очередь, обращаются к системным вызовам.

Использованные источники

1. Иртегов Д.В. Системные вызовы и библиотеки Unix System V Release 4. - Новосибирск: Новосиб. гос. ун- т., 2013. - 449 с.
2. Лав Р. Linux. Системное программирование.. - 2-е изд. - СПб.: Питер, 2014. - 448 с.